

TensorConversion.h & TensorConversion.c

Deep Teaching Guide — Every Line Explained: Syntax · Variables · Concepts · Why

Contents:

- 1 — Essential C concepts you need before reading the code
- 2 — TensorConversion.h — Header, line by line
- 3 — conversionMatrix — the dispatch table
- 4 — convertTensor — the entry point
- 5 — Helper functions: zeroTensorData, copyDimsAndSparsityToTensor
- 6 — convertFloatTensorToSymInt32Tensor (the core training conversion)
- 7 — requantSymInt32Tensor (dynamic two-pass requantisation)
- 8 — requantSymInt32TensorToScale (fixed-scale one-pass)
- 9 — INT32 conversions (→Float, →SymInt32, →Asym)
- 10 — SYM_INT32 conversions (extractInt32, →Float32, →Asym)
- 11 — ASYM conversions (→Int32, →Float, →SymInt32)
- 12 — quantTypeToString / unsupportedConversionTypes
- 13 — _Static_assert guard explained

1 — Essential C Concepts

Pointer (the * symbol) — A variable that stores the memory ADDRESS of another variable, not the value itself. When you write `tensor_t *t` it means: `t` holds the address of a `tensor_t`, not a copy of it. To reach the actual tensor you follow the address (dereference). This is how C passes large objects cheaply — you pass the address (8 bytes), not a full copy (hundreds of bytes).

Think of it as: A pointer is a sticky note that says 'the real thing is over there in room 42'.

typedef — **typedef** creates a new name (alias) for an existing type. After `typedef unsigned int myUInt;`, writing `myUInt x;` is identical to `unsigned int x;`. The compiler replaces the alias with the real type everywhere. It makes long type names shorter and code easier to read.

Think of it as: typedef is like giving your friend's phone number a contact name — you dial 'Mohamed' but the phone dials the real number.

Function pointer — In C, a function has an address in memory just like a variable. A function pointer is a variable that stores that address. You can then call whichever function the pointer holds. Syntax: `void (*fp)(int, int)` — 'fp is a pointer to a function that takes two ints and returns void'.

Think of it as: A function pointer is a phone number — you call whoever's number is stored.

cast e.g. (float)x — A cast forces a value from one type to another. `(float)inputInt32[i]` tells the compiler 'treat this integer as a float'. The value is converted. No cast = the compiler may truncate or refuse to compile.

Think of it as: Casting is like converting dollars to euros — the number changes but it represents the same real value.

size_t — An unsigned integer type used for sizes and array indices. On a 64-bit system it is 8 bytes, so it can hold values up to ~18 billion. Always use `size_t` for loop counters over arrays — it can never be negative, so it avoids sign bugs.

static (function qualifier) — When **static** is placed before a function definition at file scope, it means the function is PRIVATE to that .c file. No other .c file can call it. Used for internal helper functions that the outside world should not touch.

Think of it as: static is like an 'internal use only' label on an office tool.

memset(ptr, byte, n) — Fills `n` bytes starting at `ptr` with the value `byte`. `memset(buf, 0, 100)` writes 100 zero bytes. Used to clear a buffer before writing to it.

Think of it as: memset is like filling a whiteboard with blanks before writing new content.

memmove(dest, src, n) vs memcpy — Both copy `n` bytes from `src` to `dest`. **memcpy** is faster but UNSAFE if the source and destination memory regions overlap — it may corrupt data. **memmove** handles overlapping regions correctly by using an internal temporary buffer if needed. Rule: if in doubt whether they overlap, always use **memmove**.

Think of it as: memcpy = carrying boxes in a line without looking back. memmove = using a careful forklift that handles overlaps.

VLA — Variable Length Array — C99 allows arrays whose size is determined at runtime, not compile time. `int32_t data[n];` where `n` is a variable computed at runtime. The array lives on the STACK (not heap). Safe for small `n`. Dangerous on microcontrollers (STM32 has a small stack).
Think of it as: A VLA is like ordering exactly as many chairs as your guests — not a fixed number decided in advance.

fabsf(x) — Standard C math function. Returns the absolute value of `x` as a float. `fabsf(-3.7) = 3.7`, `fabsf(3.7) = 3.7`. The 'f' suffix means 'float version' (vs `fabs` which is double).

powf(base, exp) — Standard C math: computes base raised to the power `exp`, returning a float. `powf(2.0f, 8.0f) = 256.0f`. Used here to compute `qMax = 2^(qMaxBits-1) - 1`.

exit(1) — Immediately terminates the entire program with exit code 1 (meaning error). Used after printing an error message when a fatal condition is detected. Code after `exit(1)` never runs.

2 — TensorConversion.h — Line by Line

Here is the complete header file:

```
1  #ifndef TENSOR_CONVERSION_H
2  #define TENSOR_CONVERSION_H
3
4  #include "Tensor.h"
5
6  typedef void (*conversionFunction_t)(tensor_t *inputTensor, tensor_t *outputTensor);
7
8  void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor);
9  void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor);
10 void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor);
11 char *quantTypeToString(qtype_t t);
12 extern conversionFunction_t conversionMatrix[6][6];
13
14 #endif
```

Line 1–2 (header guard)

#ifndef TENSOR_CONVERSION_H means: 'if the symbol **TENSOR_CONVERSION_H** has NOT been defined yet'. **#define TENSOR_CONVERSION_H** then defines it. Together they form a HEADER GUARD: the first time the compiler sees this file, the condition is true and it processes everything. The second time (e.g. from another .c file), the symbol already exists, the condition is false, and the entire file is skipped. This prevents the compiler from seeing the same declarations twice, which would be an error.

The **#endif** on line 14 closes the **#ifndef** block.

Line 4 (#include "Tensor.h")

This brings in all the struct definitions that **TensorConversion.h** uses: **tensor_t**, **shape_t**, **quantization_t**, **qtype_t**, **symInt32QConfig_t**, **asymQConfig_t**, etc. Without this line the compiler would not know what **tensor_t** is and would refuse to compile. The double-quotes **"Tensor.h"** (vs **<Tensor.h>** with angle brackets) tell the compiler to look in the project's own folders first before system folders.

Line 6 (typedef — the most important line in this header)

Let's break it apart piece by piece:

typedef — 'I am creating a new type name (alias).'

void — the function that this pointer points to returns nothing (no return value).

(*conversionFunction_t) — the new type name is **conversionFunction_t**, and the * inside the parentheses means it is a **POINTER** to a function (not a pointer to a variable).

(tensor_t *inputTensor, tensor_t *outputTensor) — any function of this type must accept exactly two parameters: a pointer to the input tensor and a pointer to the output tensor.

So after this line, **conversionFunction_t** is a data type. You can declare a variable of this type: **conversionFunction_t fn;** — and **fn** will hold the address of any function that takes two **tensor_t*** and returns **void**. You can then call the function through it: **fn(input, output);**

Why create this type? Because the **conversionMatrix** table (line 12) is filled with these pointers. Without this typedef, every cell of the table would have to be written with the full ugly syntax **void (*matrix[6][6])(tensor_t*, tensor_t*)** which is unreadable.

Line 8–11 (function declarations)

These four lines declare the **PUBLIC** interface — the functions that other **.c** files are allowed to call. A **DECLARATION** tells the compiler the function's name, return type, and parameter types. The **DEFINITION** (the actual code body) lives in **TensorConversion.c**.

void convertTensor(...) — the main entry point. You call this, and it figures out which specific converter to use. You never need to call the individual conversion functions directly.

void requantSymInt32Tensor(...) — requantise a **SYM_INT32** tensor with a freshly computed scale (two-pass). Used after gradient updates in FQT training.

void requantSymInt32TensorToScale(...) — requantise with a **PRE-SET** scale that the caller already stored in the output tensor's config. One-pass, may saturate by design.

char *quantTypeToString(qtype_t t) — converts the enum value (e.g. **FLOAT32**) to the string "**FLOAT32**". The return type is **char *** (pointer to a character = a string in C). **qtype_t** is the enum type defined in **Tensor.h**.

Line 12 (extern — the conversionMatrix declaration)

extern means: 'this variable **EXISTS** somewhere else — do not allocate it here, just declare it so the compiler knows it exists.'

conversionFunction_t conversionMatrix[6][6] — **conversionMatrix** is a two-dimensional array. It has 6 rows and 6 columns. Each cell holds one **conversionFunction_t** (a function pointer). Row index = source type, column index = destination type. For example, cell **[FLOAT32][SYM_INT32]** holds the address of **convertFloatTensorToSymInt32Tensor**.

The actual array (the memory) is defined in **TensorConversion.c**. The **extern** declaration here just makes the name visible to any file that includes this header, so other modules can look up the matrix if needed.

3 — conversionMatrix — The Dispatch Table

The conversionMatrix is a 6×6 table of function pointers. Its purpose: given a source type and a destination type, immediately find the correct conversion function without writing a chain of if/else statements.

```
1  conversionFunction_t conversionMatrix[6][6] = {
2      [INT32] = {
3          [FLOAT32] = convertInt32TensorToFloatTensor,
4          [SYM_INT32] = convertInt32TensorToSymInt32Tensor,
5          [SYM] = unsupportedConversionTypes,
6          [ASYM] = convertInt32TensorToAsymTensor,
7          [BOOL] = unsupportedConversionTypes,
8      },
9      [FLOAT32] = {
10         [INT32] = convertFloatTensorToInt32Tensor,
11         [SYM_INT32] = convertFloatTensorToSymInt32Tensor,
12         [SYM] = convertFloatTensorToSymTensor,
13         [ASYM] = convertFloatTensorToAsymTensor,
14         [BOOL] = unsupportedConversionTypes,
15     },
16     [SYM_INT32] = {
17         [INT32] = extractInt32TensorFromSymInt32Tensor,
18         [FLOAT32] = convertSymInt32TensorToFloat32Tensor,
19         [SYM_INT32]= requantSymInt32Tensor,
20         [ASYM] = convertSymInt32TensorToAsymTensor,
21     },
22     [ASYM] = {
23         [INT32] = convertAsymTensorToInt32Tensor,
24         [FLOAT32] = convertAsymTensorToFloatTensor,
25         [SYM_INT32]= convertAsymTensorToSymInt32Tensor,
26     },
27 };
```

Line 1 (the array declaration)

conversionFunction_t conversionMatrix[6][6] — this is a 2D array. **[6][6]** means 6 rows and 6 columns = 36 cells total. Each cell is of type **conversionFunction_t**, which we defined as a function pointer. So each cell holds the address of a conversion function.

The **=** followed by **{}** initialises the array directly where it is declared. In C, uninitialized cells in a global array default to NULL (zero), so cells you do not fill in automatically become NULL.

Line 2 ([INT32] = { — designated initialisers)

[INT32] in square brackets inside an initialiser is called a DESIGNATED INITIALISER. It tells the compiler: 'fill row number INT32 with the values that follow.' INT32 is an enum value defined in Tensor.h as 0, so [INT32] = {...} fills row 0.

Inside that row: [FLOAT32] = convertInt32TensorToFloatTensor means 'put the address of convertInt32TensorToFloatTensor into column FLOAT32 of this row.' FLOAT32 is enum value 1, so this fills cell [0][1].

Designated initialisers make the code self-documenting — you see the type names instead of raw numbers like [0][1]. If the enum values ever change, the compiler recalculates the indices automatically.

Line 19 ([SYM_INT32][SYM_INT32] = requantSymInt32Tensor)

Notice that the diagonal of the SYM_INT32 row is NOT NULL — it is requantSymInt32Tensor. Normally, converting from a type to itself would be handled by convertTensor's same-type shortcut (which just copies bytes). But SYM_INT32 to SYM_INT32 needs special treatment: the values have changed (e.g. after a gradient update) so the scale must be recomputed. This cell ensures that if direct matrix dispatch is used, the right function is called.

★ **Why a table instead of if/else?** With 6 types there are 36 possible conversions. An if/else chain would need up to 36 branches and would grow every time you add a type. With the matrix: one lookup. Adding a new type = add one row and one column.

4 — convertTensor — The Entry Point

This is the ONLY function you call from outside. It reads the types of both tensors, then either copies data (same type) or dispatches to the correct converter.

```
1 void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     qtype_t inputDType = inputTensor->quantization->type;
3     qtype_t outputDType = outputTensor->quantization->type;
4     if (inputDType == outputDType) {
5         convertTensorsWithSameType(inputTensor, outputTensor, inputDType);
6     } else {
7         conversionFunction_t conversionFn = conversionMatrix[inputDType][outputDType];
8         if (conversionFn == NULL) {
9             PRINT_ERROR("No conversion function registered for %s to %s",
10                quantTypeToString(inputDType), quantTypeToString(outputDType));
11             exit(1);
12         }
13         conversionFn(inputTensor, outputTensor);
14     }
15 }
```

Line 1 (function signature)

void — this function returns nothing.

convertTensor — the function name.

tensor_t *inputTensor — a POINTER to the input tensor. The * means it is an address, not a copy. The input tensor can be large (hundreds of bytes) — passing a pointer is cheap (8 bytes on 64-bit).

tensor_t *outputTensor — a pointer to the output tensor. The caller must have already allocated the output tensor and set its quantization type to the desired destination type BEFORE calling this function.

Line 2–3 (reading the types)

qtype_t is an enum (a named integer type) defined in Tensor.h with values INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL.

inputTensor->quantization->type — the -> operator follows a pointer and accesses a field. So this reads: 'follow the inputTensor pointer, get its quantization field (which is itself a pointer), follow that, and read the type field.' The result is an enum value like FLOAT32.

These two lines store the source and destination types into local variables so they can be compared and used as array indices below.

Line 4–5 (same type shortcut)

If source type == destination type, no format conversion is needed. The data layout in memory is identical.

convertTensorsWithSameType handles this by just copying the raw bytes (memmove) and also copying any metadata (scale, zeroPoint) from input to output.

This is a shortcut — it avoids a matrix lookup for the common case.

Line 7 (matrix lookup)

conversionMatrix[inputDType][outputDType] — use the two enum values as row and column indices into the 6×6 array. The result is a **conversionFunction_t** — a function pointer.

The type of **conversionFn** is **conversionFunction_t**, which we defined as 'pointer to a function taking two `tensor_t*` and returning void'. So **conversionFn** now holds the address of the correct conversion function.

Line 8–11 (NULL guard)

if (conversionFn == NULL) — NULL means 'no function stored in this cell'. This should never happen if the matrix is correctly filled. But if someone accidentally looked up a combination that was never registered, we must not call NULL — that would crash with a segmentation fault (reading address 0).

Instead, print a clear error message showing WHICH types were requested (using `quantTypeToString` to convert the enum values to readable strings), then call `exit(1)` to stop the program safely.

Line 13 (calling through the pointer)

conversionFn(inputTensor, outputTensor); — this calls whichever function the pointer holds. It looks like a normal function call, but the actual function address is read from the pointer at runtime. The compiler generates a CALL instruction to the address stored in **conversionFn**. This is identical to calling the function directly — same speed, same behaviour — but the function was chosen automatically by the table lookup.

4b — convertTensorsWithSameType — Internal Copy

This is a **static** (private) helper called only by `convertTensor`. It copies bytes and metadata when source and destination types are the same.

```
1 static void convertTensorsWithSameType(tensor_t *inputTensor,
2 tensor_t *outputTensor, qtype_t qType) {
3     size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
4     size_t numberOfBytes = calcNumberOfBytesForData(inputTensor->quantization,
5     numberOfElements);
6     memmove(outputTensor->data, inputTensor->data, numberOfBytes);
7     switch (qType) {
8     case SYM_INT32: {
9         symInt32QConfig_t *inputSymIntQC = inputTensor->quantization->qConfig;
10        symInt32QConfig_t *outputSymIntQC = outputTensor->quantization->qConfig;
11        outputSymIntQC->scale = inputSymIntQC->scale;
12        break;
13    }
14    case ASYM: {
15        asymQConfig_t *inputAsymQC = inputTensor->quantization->qConfig;
16        asymQConfig_t *outputAsymQC = outputTensor->quantization->qConfig;
17        outputAsymQC->scale = inputAsymQC->scale;
18        outputAsymQC->zeroPoint = inputAsymQC->zeroPoint;
19        break;
20    }
21    default: break;
22 }
```

Line 1 (static)

static before a function at file scope makes it PRIVATE to `TensorConversion.c`. No other file can see or call this function. It is an internal helper — only `convertTensor` should call it. Using `static` prevents accidental use from other modules.

Line 3–4 (size calculations)

calcNumberOfElementsByTensor counts how many values the tensor has (e.g. a 3×4 tensor has 12 elements).

calcNumberOfBytesForData converts that count into a byte count, taking into account the quantization type. For `SYM_INT32`: each element is 4 bytes (`sizeof int32_t`). For `BOOL`: each element is 1 bit, so 8 elements = 1 byte, computed with ceiling division.

Line 5 (memmove — not memcpy)

memmove(outputTensor->data, inputTensor->data, numberOfBytes) copies `numberOfBytes` bytes from `inputTensor->data` to `outputTensor->data`.

Why **memmove** and not **memcpy**? Because the caller is allowed to pass the SAME tensor as both input and output (in-place copy). When input and output point to the same memory region, they OVERLAP. `memcpy` has undefined behaviour with overlapping regions — it may corrupt the data. `memmove` always handles overlaps correctly.

For `INT32` and `FLOAT32`, the bytes ARE the values — no scale or offset — so the copy is sufficient. For `SYM_INT32` and `ASYM`, the bytes are correct but we also need to copy the metadata (scale, `zeroPoint`).

Line 6–20 (switch for metadata)

After copying the raw bytes, we check the type to copy the quantization metadata.

For **SYM_INT32**: the `qConfig` struct contains a **scale** field (a float). We cast **qConfig** (a `void*`) to **symInt32QConfig_t*** to access it. Then we copy the scale: without this, the output tensor's scale would be random garbage.

For **ASYM**: the `qConfig` struct contains both **scale** (a float) AND **zeroPoint** (an `int16_t`). Both must be copied — the `zeroPoint` defines the zero of the asymmetric range.

For all other types (`INT32`, `FLOAT32`, `BOOL`): no metadata needed. **default: break;** exits the switch immediately.

5 — Helper Functions

zeroTensorData

```
1 void zeroTensorData(tensor_t *tensor) {
2     size_t numberOfElements = calcNumberOfElementsByTensor(tensor);
3     size_t bytesPerElement = calcBytesPerElement(tensor->quantization);
4     memset(tensor->data, 0, numberOfElements * bytesPerElement);
5 }
```

Line 1 (signature)

Takes a single tensor pointer. Returns nothing.

Line 2–3 (computing the total byte count)

calcNumberOfElementsByTensor — how many values? E.g. a 4×4 tensor = 16 elements.

calcBytesPerElement — how many bytes per element, based on the quantization type. For INT32: 4 bytes. For FLOAT32: 4 bytes. For SYM_INT32: 4 bytes. For BOOL: 1 byte (but actually 1 bit per element, handled by ceiling division elsewhere). Multiplying gives total bytes to clear.

Line 4 (memset)

memset(tensor->data, 0, numberOfElements * bytesPerElement) — fills every byte in the data buffer with 0. This is important before writing converted values into the output buffer: if any bytes are left over from previous data (garbage from stack or old computations), they would corrupt the output. Starting from zero guarantees a clean slate.

copyDimsAndSparsityToTensor

```
1 void copyDimsAndSparsityToTensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     outputTensor->shape = inputTensor->shape;
3     if (inputTensor->sparsity) {
4         memcpy(outputTensor->sparsity, inputTensor->sparsity, sizeof(sparsity_t));
5     }
6 }
```

Line 2 (shape pointer copy — shallow copy)

outputTensor->shape = inputTensor->shape — this copies the POINTER, not the struct. Both tensors now point to the same shape_t in memory. This is called a SHALLOW COPY.

Why is this safe? Because conversion functions never modify the shape — a FLOAT32 tensor and its SYM_INT32 version have exactly the same dimensions. Sharing the pointer is safe and avoids unnecessary memory allocation.

Line 3–5 (conditional sparsity copy)

if (inputTensor->sparsity) — in C, a pointer is 'truthy' if it is non-NULL. So this checks: does the input tensor have sparsity information attached? If the input has no sparsity (`sparsity == NULL`), we skip the copy.

memcpy(outputTensor->sparsity, inputTensor->sparsity, sizeof(sparsity_t)) — copies the entire sparsity struct byte-by-byte. **sizeof(sparsity_t)** gives the compiler-computed size of the struct in bytes.

6 — convertFloatTensorToSymInt32Tensor — Core Training Conversion

This is the most important conversion in the entire file. It turns floating-point weights into symmetric integers, which is how FQT training quantizes weights. The scale is computed from the actual data so no precision is lost to clamping.

```
1 void convertFloatTensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
3     float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);
4     symInt32QConfig_t *symInt32QC = outputTensor->quantization->qConfig;
5     uint8_t qMaxBits = symInt32QC->qMaxBits;
6     const float qMax = powf(2.0f, (float)qMaxBits - 1.0f) - 1.0f;
7     const float qMin = -powf(2.0f, (float)qMaxBits - 1.0f);
8     float scale;
9     if (absMax == 0.f) {
10         scale = 1.f;
11     } else {
12         scale = absMax / qMax;
13     }
14     outputTensor->quantization->qConfig->scale = scale;
15     int32_t *outputInt32 = (int32_t *)outputTensor->data;
16     float *inputFloat = (float *)inputTensor->data;
17     for (size_t i = 0; i < numberOfElements; i++) {
18         outputInt32[i] = roundByMode(
19             clamp(inputFloat[i] / scale, qMin, qMax),
20             symInt32QC->roundingMode);
21     }
22     copyDimsAndSparsityToTensor(inputTensor, outputTensor);
23 }
```

Line 2 (count elements)

numberOfElements is how many values the tensor contains. For a weight matrix of shape [64, 32], **numberOfElements** = 2048.

Line 3 (find absMax — the key step)

findAbsMaxFloat scans all **numberOfElements** float values in the buffer and returns the largest absolute value.

Absolute value means: ignore the sign. -0.85 and +0.85 both have absolute value 0.85.

Why do we need **absMax**? Because we need to choose a scale that maps the ENTIRE range of the data into the integer range [qMin, qMax]. The worst case is the largest magnitude value. If we set scale so that **absMax** maps to qMax, then every other value is smaller and fits too — with no clamping loss.

Line 4–5 (reading the output config)

`outputTensor->quantization->qConfig` is a `void*` pointer to the config struct. The cast `(symInt32QConfig_t *)` tells the compiler to treat it as a pointer to `symInt32QConfig_t`.

`qMaxBits` is the number of bits for the integers. Typical value: 8. This determines how fine or coarse the quantization is. 8 bits → 256 distinct values. 16 bits → 65536 distinct values.

Line 6–7 (computing qMax and qMin)

For a SYMMETRIC integer type with `qMaxBits` bits:

`qMax = 2^(qMaxBits-1) - 1`. For 8 bits: $2^7 - 1 = 127$.

`qMin = -2^(qMaxBits-1)`. For 8 bits: $-2^7 = -128$.

The range is `[-128, 127]` — symmetric around 0 (with 128 slightly larger on the negative side because of how two's complement integers work).

Why `(float)qMaxBits - 1.0f`? Because `qMaxBits` is `uint8_t` (integer) and `powf` expects floats. The cast converts it before the subtraction.

Line 9–13 (computing the scale)

`if (absMax == 0.f)` — edge case: if every value in the tensor is 0.0, then `absMax` is 0. We cannot divide by 0, so we set `scale = 1`. This is a safe fallback — all outputs will also be 0.

Otherwise: `scale = absMax / qMax`. This means: 'the largest float value (`absMax`) will be stored as the largest integer (`qMax`)'. For example: `absMax = 1.27`, `qMax = 127` → `scale = 0.01`. Then $1.27 / 0.01 = 127$ ✓, and $0.63 / 0.01 = 63$ ✓, and $-0.85 / 0.01 = -85$ ✓. No value ever exceeds the range. No clamping.

Line 14 (storing the scale)

`outputTensor->quantization->qConfig->scale = scale` — we store the computed scale into the output tensor's config. This is ESSENTIAL. Without storing the scale, whoever reads this tensor later would have no way to dequantize it (convert integers back to floats). The scale is the key that unlocks the stored values.

Line 15–16 (pointer casts for efficient access)

`int32_t *outputInt32 = (int32_t *)outputTensor->data` — `tensor->data` is `uint8_t*` (raw bytes). We cast it to `int32_t*` to treat the buffer as an array of 4-byte integers. This is valid because `SYM_INT32` stores each value as exactly one `int32_t` (4 bytes).

`float *inputFloat = (float *)inputTensor->data` — similarly, cast the input raw bytes to `float*`, because `FLOAT32` stores each value as one float (also 4 bytes).

After these casts, `inputFloat[i]` reads the *i*-th float, and `outputInt32[i]` writes to the *i*-th `int32` — clean and fast.

Line 17–21 (the quantisation loop — the heart of the function)

For each element i:

inputFloat[i] / scale — divide the float by the scale. This converts from the float world to the integer world.
Example: $0.85 / 0.01 = 85.0$.

clamp(..., qMin, qMax) — ensure the result is within [qMin, qMax]. In theory this never triggers (because $\text{scale} = \text{absMax}/\text{qMax}$, so no value exceeds qMax). But floating-point rounding could produce values like 127.0000001, so clamp is there as a safety net.

roundByMode(..., roundingMode) — convert the float to an integer. HALF_AWAY rounds 0.5 away from zero (standard rounding). SR_HALF_AWAY (stochastic) adds a tiny random jitter before rounding — this eliminates rounding bias over many training iterations, which is critical for FQT training.

outputInt32[i] = ... — store the resulting integer. It is an `int32_t`, so it can hold values from about -2 billion to +2 billion. But for 8-bit quantisation, the values will always be in [-128, 127].

→ **Complete numeric example:** float values = [1.27, -0.85, 0.50, -0.13] | qMaxBits = 8 → qMax=127 | absMax=1.27 | scale=1.27/127=0.01 | quantised = [127, -85, 50, -13] | stored scale=0.01 | to recover: [127*0.01=1.27, -85*0.01=-0.85, 50*0.01=0.50, -13*0.01=-0.13] ✓

7 — requantSymInt32Tensor — Dynamic Two-Pass Requantisation

This function re-quantises a SYM_INT32 tensor into another SYM_INT32 tensor, computing a FRESH scale from the current data. Used after a gradient update in FQT training, when the weight magnitudes have changed and the old scale no longer fits well.

```
1 void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
3     symInt32QConfig_t *inputSymInt32QC = inputTensor->quantization->qConfig;
4     symInt32QConfig_t *outputSymInt32QC = outputTensor->quantization->qConfig;
5     float inScale = inputSymInt32QC->scale;
6     const float qMax = powf(2.0f, (float)outputSymInt32QC->qMaxBits - 1.0f) - 1.0f;
7     const float qMin = -powf(2.0f, (float)outputSymInt32QC->qMaxBits - 1.0f);
8     int32_t *inputInt32 = (int32_t *)inputTensor->data;
9     int32_t *outputInt32 = (int32_t *)outputTensor->data;
10    /* --- Pass A: find absMax over dequantised values (read-only) --- */
11    float absMax = 0.f;
12    for (size_t i = 0; i < numberOfElements; i++) {
13        float dequant = fabsf((float)inputInt32[i] * inScale);
14        if (dequant > absMax) absMax = dequant;
15    }
16    float scale;
17    if (absMax == 0.f) {
18        scale = 1.f;
19    } else {
20        scale = absMax / qMax;
21    }
22    outputSymInt32QC->scale = scale;
23    /* --- Pass B: re-quantise using the new scale --- */
24    for (size_t i = 0; i < numberOfElements; i++) {
25        outputInt32[i] = roundByMode(
26            clamp(((float)inputInt32[i] * inScale) / scale, qMin, qMax),
27            outputSymInt32QC->roundingMode);
28    }
29 }
```

Line 3–4 (getting the quantisation configs)

inputSymInt32QC = 'input symmetric-int32 quantisation config'. This struct holds the scale (and other settings) that was used when the input tensor was quantised.

outputSymInt32QC = the config for the output tensor. It provides qMaxBits (how many bits to use) and roundingMode. After the function runs, its scale field will be filled with the newly computed scale.

Line 5 (latching inScale — CRITICAL for in-place safety)

float inScale = inputSymInt32QC->scale — this copies the input scale into a local float variable.

WHY? Because the caller is allowed to call this function with `inputTensor == outputTensor` (in-place: the same tensor is both input and output). When they are the same, `inputSymInt32QC` and `outputSymInt32QC` point to THE SAME config struct.

In Pass B (line 21): **outputSymInt32QC->scale = scale** OVERWRITES the scale in that config. If we had not saved `inScale` to a local variable, Pass B would read the NEW scale instead of the original one — corrupting every element.

By saving it in the local variable **inScale** on line 5, we guarantee that even after `outputSymInt32QC->scale` is overwritten, we still have the correct original scale.

Line 10–15 (Pass A — read-only scan)

Pass A scans ALL elements to find the largest real (dequantised) value.

float dequant = fabsf((float)inputInt32[i] * inScale) — for each element:

1. Read the stored integer: `inputInt32[i]` (e.g. 108)
2. Cast to float: `(float)inputInt32[i] = 108.0f`
3. Multiply by `inScale` to dequantise: $108.0f * 0.0079f \approx 0.853f$
4. `fabsf()` takes the absolute value (so -108 and +108 both give the same `absMax`)

After the loop, **absMax** holds the largest dequantised magnitude across the entire tensor.

Pass A is READ-ONLY on the data — it never writes. This is what makes in-place use safe: Pass A finishes reading everything before Pass B starts writing.

Line 16–21 (computing and storing the new scale)

Same logic as `convertFloatTensorToSymInt32Tensor`: if `absMax == 0`, use `scale=1`. Otherwise, `scale = absMax / qMax`.

outputSymInt32QC->scale = scale — write the new scale into the output config. This overwrites the old scale. From this moment, the config represents the new quantisation.

Line 23–27 (Pass B — re-quantise with the new scale)

For each element i:

(float)inputInt32[i] * inScale — dequantise from the OLD scale. We use **inScale** (the saved local variable), NOT `outputSymInt32QC->scale` (which was just overwritten). This is why saving it on line 5 was critical.

/ scale — re-quantise using the NEW scale. This converts: `dequantised_float / new_scale = new_integer`.

clamp(..., qMin, qMax) — safety net for floating-point rounding edge cases. In practice, since `scale = absMax/qMax`, no value should exceed `qMax`. Clamping is there just in case.

roundByMode(..., roundingMode) — convert the float to an integer.

outputInt32[i] = ... — write the new integer. When in-place (`inputInt32 == outputInt32`), this overwrites the same cell that was just read. This is safe because we read element `i` in Pass A, and now in Pass B we write element `i` — we never read element `i` again after writing it, so no corruption occurs.

■ **Why two passes?** You need the new scale before you can write even a single output value. But to compute the new scale you must read ALL values first (to find `absMax`). One pass is impossible: you cannot read-and-write-simultaneously.

→ **Numeric example:** Input mantissas=[100, -80, 50, 127], inScale=0.01 | Pass A: real values=[1.00, 0.80, 0.50, 1.27], absMax=1.27, newScale=1.27/127=0.01 | Pass B: [100*0.01/0.01=100, -80*0.01/0.01=-80, 50*0.01/0.01=50, 127*0.01/0.01=127] | Output mantissas=[100, -80, 50, 127], scale=0.01 (same scale, values preserved)

8 — requantSymInt32TensorToScale — Fixed-Scale One-Pass

Same idea as `requantSymInt32Tensor`, but the target scale is set by the CALLER before calling. The function never changes the scale — it only uses it. Values that exceed the representable range are CLAMPED (saturated). This is intentional.

```
1 void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor) {
2     symInt32QConfig_t *inputSymInt32QC = inputTensor->quantization->qConfig;
3     symInt32QConfig_t *outputSymInt32QC = outputTensor->quantization->qConfig;
4     float inScale = inputSymInt32QC->scale;
5     float targetScale = outputSymInt32QC->scale;
6     if (!(targetScale > 0.f)) {
7         PRINT_ERROR("requantSymInt32TensorToScale: target scale must be pre-set and > 0");
8         exit(1);
9     }
10    const float qMax = powf(2.0f, (float)outputSymInt32QC->qMaxBits - 1.0f) - 1.0f;
11    const float qMin = -powf(2.0f, (float)outputSymInt32QC->qMaxBits - 1.0f);
12    int32_t *inputInt32 = (int32_t *)inputTensor->data;
13    int32_t *outputInt32 = (int32_t *)outputTensor->data;
14    for (size_t i = 0; i < numberOfElements; i++) {
15        outputInt32[i] = roundByMode(
16            clamp(((float)inputInt32[i] * inScale) / targetScale, qMin, qMax),
17            outputSymInt32QC->roundingMode);
18    }
19 }
```

Line 4–5 (latching both scales)

inScale — the input tensor's current scale (same latch trick as `requantSymInt32Tensor`: save it NOW before any writes happen, for in-place safety).

targetScale — the target scale. The CALLER must write this into `outputSymInt32QC->scale` BEFORE calling this function. The function reads it but NEVER modifies it.

Line 6 (NaN-robust guard — the subtlety here)

if (!(targetScale > 0.f)) — note the double negation. Why not simply **if (targetScale <= 0.f)**?

Because of NaN (Not a Number). In IEEE 754 floating point, NaN compared to anything returns false. So **NaN <= 0.f** is FALSE — the guard would not trigger, and we would proceed with a NaN scale, producing garbage output.

!(targetScale > 0.f) is true when `targetScale <= 0` OR when `targetScale` is NaN, because **NaN > 0.f** is also FALSE, making **!(FALSE) = TRUE**.

This single condition safely catches both negative scales AND NaN.

Line 14–18 (the single-pass loop)

Only ONE pass needed — because the scale is pre-set, we do not need to scan all values first.

For each element: dequantise with `inScale`, then re-quantise with `targetScale`.

clamp — here, clamping IS expected and intentional. If a value in the input is larger than `targetScale * qMax`, it gets clamped to `qMax` (or `qMin`). This is called SATURATION. It matches the Deutel FQT paper (Equation 4). The caller chose the target scale deliberately; the function honours it.

★ **Key difference from `requantSymInt32Tensor`:** `requantSymInt32Tensor` COMPUTES the scale from the data (no clamping ever). `requantSymInt32TensorToScale` USES a pre-set scale (clamping by design). Use `requantSymInt32Tensor` after weight updates. Use `requantSymInt32TensorToScale` when the scale is fixed by the training algorithm.

9 — INT32 Conversions

convertInt32TensorToFloatTensor

```
1 void convertInt32TensorToFloatTensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
3     int32_t inputData[numberOfElements];
4     float outputData[numberOfElements];
5     readBytesAsInt32Array(numberOfElements, inputTensor->data, inputData);
6     zeroTensorData(outputTensor);
7     for (size_t i = 0; i < numberOfElements; i++) {
8         outputData[i] = (float)inputData[i];
9     }
10    writeFloatArrayToByteArray(numberOfElements, outputData, outputTensor->data);
11    copyDimsAndSparsityToTensor(inputTensor, outputTensor);
12 }
```

Line 3–4 (VLAs — stack arrays of runtime size)

int32_t inputData[numberOfElements] — a Variable Length Array. numberOfElements is only known at runtime. C99 allows this on the stack. For large tensors on STM32, this could overflow the stack — be careful.

float outputData[numberOfElements] — same, for the converted floats.

Line 5 (readBytesAsInt32Array)

The raw bytes in tensor->data are not directly usable as int32_t on some architectures (alignment issues). This helper safely decodes them into a proper int32_t array.

Line 8 ((float) cast — no scale involved)

(float)inputData[i] — INT32 tensors have no scale. They store raw integers. Converting to float just changes the representation: the integer 127 becomes the float 127.0f. No scale multiplication — the value is identical. This is NOT dequantisation; it is just a type conversion.

convertInt32TensorToSymInt32Tensor

```
1 void convertInt32TensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
3     symInt32QConfig_t *outputSymInt32QConfig = outputTensor->quantization->qConfig;
4     outputSymInt32QConfig->scale = 1;
5     memcpy(outputTensor->data, inputTensor->data, numberOfElements * sizeof(int32_t));
6 }
```

Line 4 (scale = 1)

INT32 has no scale concept — the values are raw integers. By setting scale = 1, we say: 'real value = mantissa × 1 = mantissa.' The integer values are unchanged. We just wrap them in the SYM_INT32 format.

Line 5 (memcpy — direct byte copy)

INT32 and SYM_INT32 both store values as 4-byte `int32_t`. The bytes are identical. So we can copy directly without any conversion. **`sizeof(int32_t)`** = 4 bytes, so total = `numberOfElements * 4` bytes.

10 — SYM_INT32 Conversions

extractInt32TensorFromSymInt32Tensor

```
1  /* Important: Scale is ignored! */
2  void extractInt32TensorFromSymInt32Tensor(tensor_t *inputTensor, tensor_t
   *outputTensor) {
3      size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
4      int32_t inputAsInt32[numberOfElements];
5      readBytesAsInt32Array(numberOfElements, inputTensor->data, inputAsInt32);
6      memcpy(outputTensor->data, inputAsInt32, numberOfElements * sizeof(int32_t));
7  }
```

Line 1 (scale is ignored — intentional)

This function extracts the raw integer mantissas. It does NOT multiply by scale — it does NOT dequantise. You get the stored integers as-is. If you want the real float values, use `convertSymInt32TensorToFloat32Tensor` instead.

When is this useful? When your algorithm needs to work with the raw integers directly, e.g. to inspect or manipulate the quantised representation.

convertSymInt32TensorToFloat32Tensor

```
1  void convertSymInt32TensorToFloat32Tensor(tensor_t *inputTensor, tensor_t
   *outputTensor) {
2      int32_t *inputAsInt32 = (int32_t *)inputTensor->data;
3      symInt32QConfig_t *inputSymInt32QConfig = inputTensor->quantization->qConfig;
4      float scale = inputSymInt32QConfig->scale;
5      float output[numberOfValues];
6      for (size_t i = 0; i < numberOfValues; i++) {
7          output[i] = (float)inputAsInt32[i] * scale;
8      }
9      memcpy(outputTensor->data, output, numberOfValues * sizeof(float));
10 }
```

Line 7 (dequantisation formula)

(float)inputAsInt32[i] * scale — this is the inverse of quantisation.

When we quantised: $\text{integer} = \text{round}(\text{float} / \text{scale})$

To recover: $\text{float} \approx \text{integer} \times \text{scale}$

Example: stored integer = 108, scale = 0.0079 $\rightarrow 108 \times 0.0079 = 0.853 \approx$ original 0.85.

The result is approximate — some precision was lost when rounding during quantisation. This is the fundamental trade-off of quantisation.

11 — ASYM Conversions

ASYM (asymmetric) quantisation uses two parameters: scale and zeroPoint. The formula is: **real_value = (stored_integer + zeroPoint) × scale**. This allows the representable range to be shifted — useful for post-ReLU activations (which are always ≥ 0).

convertAsymTensorToInt32Tensor

```
1 void convertAsymTensorToInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     asymQConfig_t *asymQConfig = inputTensor->quantization->qConfig;
3     int16_t zeroPoint = asymQConfig->zeroPoint;
4     uint8_t dataOut[numberOfElements * sizeof(int32_t)];
5     memset(dataOut, 0, numberOfElements * sizeof(int32_t));
6     byteConversion(inputTensor->data, asymQConfig->qBits, dataOut, 32, numberOfElements);
7     int32_t outputElements[numberOfElements];
8     readBytesAsInt32Array(numberOfElements, dataOut, outputElements);
9     for (size_t elementIndex = 0; elementIndex < numberOfElements; elementIndex++) {
10        outputElements[elementIndex] = outputElements[elementIndex] + zeroPoint;
11    }
12    writeInt32ArrayToByteArray(numberOfElements, outputElements, outputTensor->data);
13    copyDimsAndSparsityToTensor(inputTensor, outputTensor);
14 }
```

Line 3 (zeroPoint — what is it?)

zeroPoint is an integer stored in the ASYM config. It represents where 'zero' lives in the unsigned integer range. When the original float was 0.0, the stored integer is... not necessarily 0. It is zeroPoint. So to convert back from stored integer to signed integer: $\text{signed_int} = \text{stored_int} + \text{zeroPoint}$.

Line 6 (byteConversion — unpacking packed bits)

ASYM tensors store values in qBits bits each (e.g. 8 bits). The raw buffer stores them packed together — not padded to 32 bits. **byteConversion(input, qBits, output, 32, n)** expands each qBits-wide value into a 32-bit integer in the output buffer. This is the 'unpacking' step.

After this call, dataOut contains n integers, each occupying 4 bytes.

Line 10 (adding zeroPoint)

outputElements[i] = outputElements[i] + zeroPoint — This converts from the unsigned ASYM integer (always positive, range [0, qMax]) to a signed integer centred around the original data's range. Without this step the result would be wrong — all values would be shifted.

Example: if scale=0.01, zeroPoint=-128, stored_int=200 → signed = 200 + (-128) = 72 → real $\approx 72 * 0.01 = 0.72$.

convertAsymTensorToFloatTensor

```
1 void convertAsymTensorToFloatTensor(tensor_t *inputTensor, tensor_t *outputTensor) {
2     /* unpack qBits integers to int32 in inputInt array... */
```

```

3 float *outputElements = (float *)outputTensor->data;
4 for (size_t i = 0; i < numberOfElements; i++) {
5     outputElements[i] = ((float)inputInt[i] + (float)zeroPoint) * asymQConfig->scale;
6 }
7 copyDimsAndSparsityToTensor(inputTensor, outputTensor);
8 }

```

Line 5 (full dequantisation formula)

$((\text{float})\text{inputInt}[i] + (\text{float})\text{zeroPoint}) * \text{scale}$ — this is the ASYM dequantisation formula.

Step 1: $(\text{float})\text{inputInt}[i]$ — cast the unpacked integer to float.

Step 2: $+ (\text{float})\text{zeroPoint}$ — shift by the zero-point offset. This converts from the asymmetric $[0, \text{qMax}]$ range back to the centred range.

Step 3: $\times \text{scale}$ — scale back to the original float range.

Example: $\text{stored_int}=50, \text{zeroPoint}=-128, \text{scale}=0.01 \rightarrow (50 + (-128)) \times 0.01 = -78 \times 0.01 = -0.78$.

12 — quantTypeToString / unsupportedConversionTypes

quantTypeToString

```
1 char *quantTypeToString(qtype_t t) {
2     switch (t) {
3         case INT32: return "INT32";
4         case FLOAT32: return "FLOAT32";
5         case SYM_INT32: return "SYMINT32";
6         case SYM: return "SYM";
7         case ASYM: return "ASYM";
8         case BOOL: return "BOOL";
9         default: return "UNKNOWN";
10    }
11 }
```

Line 1 (return type: char *)

char * is a pointer to a character. In C, a string is just a sequence of characters ending with a null byte (`\0`). So returning **char *** means returning a STRING.

The string literals like **"INT32"** are stored in read-only memory at compile time. Returning a pointer to them is safe — they live for the entire program duration.

qtype_t t is the enum value to convert. Enum values are just integers (`INT32=0`, `FLOAT32=1`, etc.) but switch/case works on their symbolic names.

Line 9 (default: return "UNKNOWN")

The switch covers all 6 defined enum values. The default branch catches any value that is not in the enum — this should never happen in correct code, but adding it makes the function robust. Without it, the compiler may warn about not handling all values.

unsupportedConversionTypes

```
1 void unsupportedConversionTypes(tensor_t *inputTensor, tensor_t *outputTensor) {
2     qtype_t inputQType = inputTensor->quantization->type;
3     qtype_t outputQType = outputTensor->quantization->type;
4     PRINT_ERROR("Conversion from %s to %s is not supported",
5     quantTypeToString(inputQType), quantTypeToString(outputQType));
6     exit(1);
7 }
```

Line 1 (why does this exist?)

The conversionMatrix must have a function in EVERY cell — even combinations that are not supported. If a cell were left as NULL, and convertTensor somehow reached it (bypassing the NULL check), the program would crash with a null pointer dereference — no error message, just a crash.

By filling unsupported cells with unsupportedConversionTypes, the program fails CLEARLY: it prints exactly which conversion was attempted and then stops.

Line 4–5 (PRINT_ERROR with quantTypeToString)

PRINT_ERROR is a macro defined in Common.h that prints the file name, line number, and the message. The %s format specifiers are filled by quantTypeToString calls, producing messages like 'Conversion from FLOAT32 to BOOL is not supported'.

13 — `_Static_assert` Guard

```
1  _Static_assert(BOOL + 1 == 6,  
2  "extend conversionMatrix when adding qtype_t entries");
```

Line 1-2 (what this does)

`_Static_assert` is a C11 keyword (also spelled `static_assert`). It checks a condition AT COMPILE TIME — before the program runs. If the condition is false, the compiler refuses to build and prints the message as an error.

BOOL + 1 == 6 — `BOOL` is the last enum value in `qtype_t`. The enum has 6 values: `INT32=0`, `FLOAT32=1`, `SYM_INT32=2`, `SYM=3`, `ASYM=4`, `BOOL=5`. So `BOOL+1 = 6`. This condition is true, so the build succeeds.

Now suppose someone adds a new type `FP8` to `qtype_t`. The compiler reassigns `BOOL=6` (`FP8` may be 5 or 6). `BOOL+1 = 7 ≠ 6`. The condition is false. The compiler REFUSES to build, showing the message: 'extend conversionMatrix when adding qtype_t entries'. The developer MUST add a new row and column to the matrix before the code will compile.

Without this guard: adding `FP8` would compile silently, but then `conversionMatrix[FP8][...]` would be an out-of-bounds access (index 6 in a `[6][6]` array). That is undefined behaviour — silent memory corruption, random crashes. `_Static_assert` turns a hard-to-debug runtime bug into an obvious compile error.

→ **`_Static_assert` vs `assert()`**: `assert()` checks at RUNTIME (when the program is running) and crashes if false. `_Static_assert` checks at COMPILE TIME and fails before the program is even built. For things that can be known at compile time (like enum sizes), `_Static_assert` is always better.